K-Nearest Neighbours (KNN)

Contents

1. Overview . . . 2
2. How KNN predicts . . . 2
   - 2.1 The steps . . . 2
   - 2.2 Any number of features . . . 5
3. KNN in scikit-learn: the breast cancer data . . . 5
   - 3.1 The data . . . 5
   - 3.2 Split and scale . . . 5
   - 3.3 Train, predict, score . . . 7
4. Choosing k . . . 8
   - 4.1 The rule of thumb: square root of n . . . 8
   - 4.2 Experiments: try every k . . . 9
5. Decision surfaces . . . 11
   - 5.1 What it shows . . . 11
   - 5.2 How it is built . . . 12
6. How k controls overfitting and underfitting . . . 13
   - 6.1 k = 1: overfitting . . . 14
   - 6.2 k = n: underfitting . . . 14
   - 6.3 The middle . . . 15
   - 6.4 Trying it: the k slider app . . . 15
7. When KNN fails . . . 15
   - 7.1 Large datasets: slow predictions . . . 16
   - 7.2 High-dimensional data: distances stop meaning much . . . 17
   - 7.3 Outliers . . . 17
   - 7.4 Imbalanced data . . . 17
   - 7.5 Inference: which feature mattered? . . . 18
8. Summary . . . 19
9. Sources . . . 19
10. Key terms . . . 20

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the Course map.



- **Builds on:** Classification problems; Instance-based learning; Vector magnitude, distance and scalar operations; Feature scaling; Cross-validation; Curse of dimensionality.
- **Leads to:** Decision trees; SMOTE; Nearest neighbours with wrap-around.
- **Compare with:** Overfitting; Bias-variance trade-off; ANN for classification.

1. Overview

Key point: KNN predicts a new point's class from the classes of the k training points closest to it. KNN is simple and often accurate, but slow on big data and unreliable when distances stop meaning much.

K-nearest neighbours (KNN) (G-998) is a classification algorithm built on one idea: a point is probably like its **neighbours** (G-1306). A popular saying puts it the same way: "you are the average of the five people you spend the most time with". Despite its simplicity, KNN often comes surprisingly close to the best possible classifier (ISL §2.2.3).

This Note covers how KNN predicts, how to run it in scikit-learn, how to choose k, what k does to the **decision surface** (G-560), and the six situations where KNN works badly. The Notebook (`ML-085-knn.ipynb`) runs every example, and `app.py` lets us move k with a slider.

2. How KNN predicts

Key point: Choose k, measure the distance from the new point to every training point, keep the k nearest, and take a majority vote.

2.1 The steps

Key point: Distance, sort, keep k, vote. Nothing else.

Instance-based learning (G-955) already showed one KNN prediction step by step on the student data. Each student is one **observation** (G-1374; one record, a row of the data table). Each has two **features** (G-772; input variables, one column each), CGPA and IQ, and a **target** (G-1949; the output we predict): the placement result (1 = placed, 0 = not placed). A new student, the **query point** (G-1605), arrives with a known CGPA and IQ, and we must predict placement.

KNN does it in five steps:

1. **Choose k**, the number of neighbours to consult, for example k = 3.

2. **Measure** the distance from the query point to every training point. With 100 students, that is 100 distances. The usual choice is the **Euclidean distance** (G-715; the **KNN imputer** (G-1017), section 4.1): the square root of the summed squared differences, feature by feature.
3. **Sort** the distances from smallest to largest.
4. **Keep the k nearest** training points: the **neighbours** (G-1306).
5. **Vote** (a **majority vote**, G-1146): each neighbour "says" its class, and the class with the most votes wins. If the 3 neighbours say 1, 1 and 0, the prediction is 1 (placed).

The steps on four students. A tiny table of our own, to make step 2 concrete. Each row is one training student: CGPA, IQ and the placement result.

Student	CGPA	IQ	Placed
A	8.0	120	1
B	5.5	90	0
C	7.5	110	1
D	6.0	95	0

The four students below are invented to keep the arithmetic small. The query student has CGPA 7.0 and IQ 105. The Euclidean distance to a training student is: subtract each feature, square it, add the squares, take the square root. Student A, one line per step:

$$\text{CGPA difference} = 7.0 - 8.0 = -1.0$$

$$(-1.0)^2 = 1.00$$

$$\text{IQ difference} = 105 - 120 = -15$$

$$(-15)^2 = 225$$

$$\text{sum of squares} = 1.00 + 225 = 226$$

$$\text{distance} = \sqrt{226} = 15.03$$

The same steps for the other students give the distances in this table (step 2), already sorted (step 3):

Student	CGPA difference squared	IQ difference squared	Sum	Distance	Placed
C	0.25	25	25.25	5.02	1
D	1.00	100	101.00	10.05	0
A	1.00	225	226.00	15.03	1
B	2.25	225	227.25	15.08	0

With k = 3, the neighbours (step 4) are C, D and A. Their votes (step 5) are 1, 0 and 1: two against one, so the prediction is 1 (placed). Notice that the IQ column supplies almost all of every distance, because IQ numbers are far bigger than CGPA numbers. Section 3.2 fixes this.

Figure 1 runs these steps on one test tumour from the breast cancer data of section 3 (two scaled features, k = 5). Watch the circle grow until it holds exactly 5 training tumours; the vote is then 3 benign to 2 malignant, so the prediction is benign.

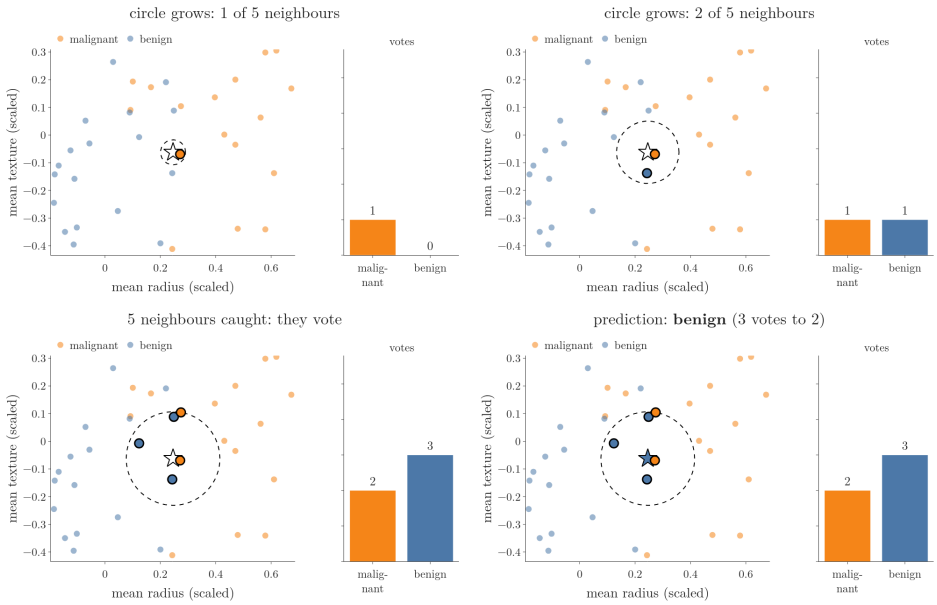


Figure 1: KNN with k = 5: a circle grows around the query tumour (star) until it holds 5 neighbours, which then vote. Placing a new point among labelled points and letting its k nearest neighbours vote follows StatQuest's "K-nearest neighbors, Clearly Explained" (Starmer).

> **Extra:** Euclidean is not the only distance. scikit-learn's `KNeighborsClassifier` uses the **Minkowski distance** (G-1227) with `p=2` by default, which is exactly the Euclidean distance (scikit-learn docs, `KNeighborsClassifier`). With `p=1` it becomes the **Manhattan distance**: the sum of the absolute differences, like walking along city blocks.

2.2 Any number of features

> **Key point:** Every observation is a point (a vector) with one coordinate per feature, so KNN works the same way in 3, 30 or 300 dimensions.

Nothing in the steps needs exactly two features. With 30 features, every observation is a point in 30-dimensional space, that is, a **vector** of 30 numbers. The Euclidean distance simply gets one squared term per feature.

We cannot draw 30 dimensions, but the computer does not need to. KNN treats the **query point** (G-1605) as a vector, computes its distance to every training vector, keeps the k closest, and votes.

3. KNN in scikit-learn: the breast cancer data

> **Key point:** Split, scale (fit on the training set only), fit `KNeighborsClassifier(n_neighbors=5)`, predict, score: 97.4% test accuracy.

3.1 The data

> **Key point:** 569 tumours, 30 numeric features each, and a diagnosis: malignant or benign.

The **breast cancer dataset** describes 569 breast tumours. Each has 30 numeric features, measurements of the cell nuclei such as `radius_mean`, `texture_mean` and `compactness_mean`. The target, the **diagnosis**, is malignant (cancer, 212 tumours) or benign (not cancer, 357 tumours).

The widely shared CSV version also has an `id` column and an empty last column, `Unnamed: 32`. Neither helps predict anything, so both are dropped. scikit-learn ships a cleaned copy without them, which we use.

> **Python:** Loading the data.

```python
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target    # 30 inputs; 0 = malignant, 1 = benign
X.shape                           # (569, 30)
```

`as_frame=True` returns pandas tables instead of plain arrays. No values are missing, so no cleaning is needed.

3.2 Split and scale

> **Key point:** KNN compares distances, so every feature must be on the same scale; otherwise the features with big numbers decide alone.

We hold back 20% of the observations as a test set (the train-test split). The split leaves 455 training observations and 114 test observations.

The features have very different ranges. `radius_mean` is in the tens (14, 11, 19, ...), while `compactness_mean` is a small decimal. In a Euclidean distance, a feature measured in tens outweighs a feature measured in hundredths, so the small-number features would hardly count. An everyday picture: comparing two houses by price in rupees and by number of rooms, the price differences (lakhs) swamp the room differences (one or two).

We fix this with **standardization** (G-1874; the **standardization** (G-1874)): each value has the feature's mean subtracted and is then divided by the feature's standard deviation, so every feature ends with mean 0 and standard deviation 1. A small instance with invented numbers, three houses with prices 40, 60 and 80 lakh rupees (mean 60, standard deviation 20) and 2, 3 and 4 rooms (mean 3, standard deviation 1):

House	Price	(Price - 60) / 20	Rooms	(Rooms - 3) / 1
1	40	-1	2	-1
2	60	0	3	0
3	80	1	4	1

Both features now run from -1 to 1. The distance between houses 1 and 2 shows the effect, one step per line:

Before scaling:

$$\sqrt{(40-60)^2+(2-3)^2}$$

$$=\sqrt{400+1}=20.02$$

After scaling:

$$\sqrt{(-1-0)^2+(-1-0)^2}$$

$$=\sqrt{1+1}=1.41$$

Before scaling, the room difference hardly counts; after scaling, price and rooms count equally. The formula for any value x is:

$$z=\frac{x-\text{mean}}{\text{standard deviation}}$$

For the price of house 1:

$$(40 - 60)/20 = -1$$

As always, the scaler learns the mean and standard deviation from the training set only and then applies them to the test set; fitting it on the test set would leak test information into training (see scaling the inputs).

Python: Splitting and scaling.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=2)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)  # learn mean, std
X_test_s = scaler.transform(X_test)        # reuse them
```

The result is a NumPy array, no longer a pandas table, still with 455 rows and 30 columns.

3.3 Train, predict, score

Key point: With k = 5, KNN gets 111 of the 114 test tumours right: 97.4%.

Python: KNN in scikit-learn.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=5)  # 5 is the default
knn.fit(X_train_s, y_train)
y_pred = knn.predict(X_test_s)
accuracy_score(y_test, y_pred)             # 0.974
```

`n_neighbors` is k: the size of the neighbourhood.

To score a classifier we use **accuracy** (G-162; the accuracy): correct predictions divided by all predictions. For example, 180 correct out of 200 test students gives:

$$180/200 = 0.90$$

That is 90%. Here, 111 of 114 correct gives 97.4%.

Without scaling, the same model gets only 104 of 114 right (91.2%). The gap of 6 points is the cost of letting the big-number features dominate the distances.

Figure 2 shows where the gap comes from. Its left panel has a **log scale** on the vertical axis: each gridline is ten times the one below it (0.01, 0.1, 1, 10, 100, 1000), so bars that differ by a factor of 100,000 fit on one chart. Before scaling, the spread of the 30 features runs from

about 0.002 to over 500: `worst area` alone varies more than a hundred thousand times more than the smallest feature, so it decides most distances. After **standardization** (G-1874) every feature has a spread of 1 and gets an equal say.

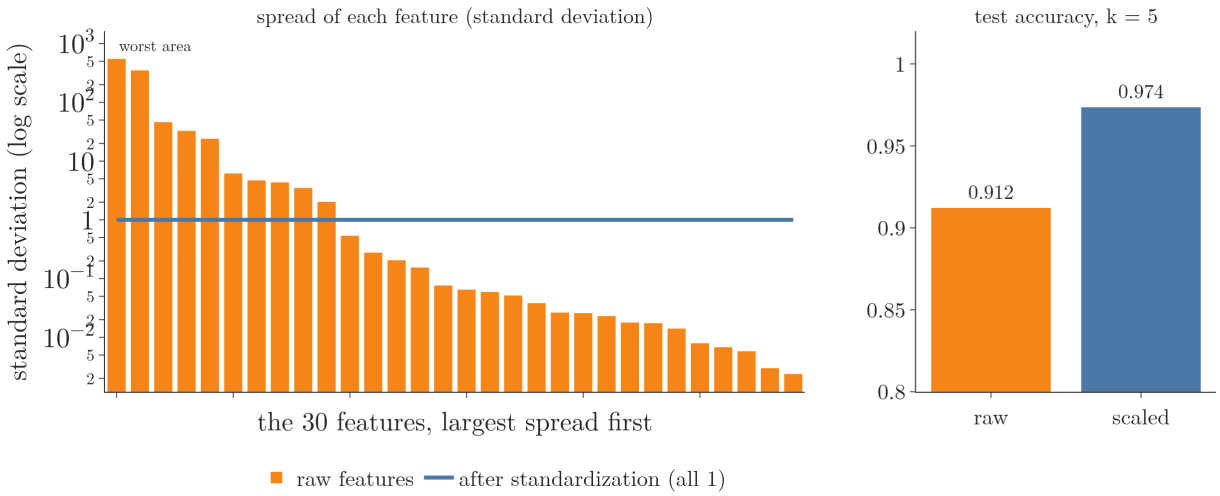


Figure 2: Left: the standard deviation of each of the 30 breast cancer features on the training set, raw (orange bars, log scale) and after standardization (blue line, all 1). Right: test accuracy with k = 5, raw 0.912 and scaled 0.974.

4. Choosing k

> **Key point:** There is no universally right k. A rough start is $\sqrt{n}$, made odd; the reliable way is to try many values with **cross-validation** (G-510).

k (`n_neighbors`, G-992) is a **hyperparameter** (G-910): a setting we choose before training (the **pipelines** (G-1499)). Different datasets need different k, so we must find a good value for each one. There are two approaches: a rule of thumb and experiments.

4.1 The rule of thumb: square root of n

> **Key point:** $k \approx \sqrt{n}$, where n is the number of training observations; then pick a nearby odd number to avoid ties.

1. **In words:** take the square root of the number of training observations.
2. **Formula:**

$$k \approx \sqrt{n}$$

3. **Example:** with 400 patients:

$$\sqrt{400} = 20$$

On our breast cancer data:

$$\sqrt{455} \approx 21.3$$

So k is about 21.

Why the square root: theory says k should grow as the training set grows, so each vote averages more **neighbours** (G-1306), but stay a tiny fraction of n, so the neighbours stay close to the new point. With both, KNN's error approaches the best error possible as n grows (Anava and Levy 2016, §1). The square root does both:

n	$k = \sqrt{n}$	k as a share of n
400	20	5%
10,000	100	1%

With two classes, an even k can produce a tie. With k = 4, the vote can end 2 against 2, and there is no majority. So we take an odd value nearby: 19 or 21 instead of 20.

> **Extra:** If a tie does happen, scikit-learn does not flip a coin. The classifier picks the class that comes first in its sorted list of classes (here, 0 = malignant), as a small test in the notebook shows. An odd k avoids the question entirely for two classes.

The square-root rule is only a starting point: on a dataset of fixed size it often gives poor results, so k is usually chosen by **cross-validation** (G-510) (Anava and Levy 2016, §1). The experiments below test many values of k directly.

4.2 Experiments: try every k

> **Key point:** Train one model per k from 1 to 15, score each, and keep the best.

The idea is simple: build KNN models with k = 1, 2, 3, . . . , 15, train each on the training set, score each, and keep the k with the highest accuracy.

> **Python:** One model per k, scored on the test set.
> ```
> test_acc = []
> for k in range(1, 16):
>     m = KNeighborsClassifier(n_neighbors=k)
>     m.fit(X_train_s, y_train)
>     test_acc.append(m.score(X_test_s, y_test))
> ```
> `m.score` computes the accuracy directly, the same as `accuracy_score(y_test, m.predict(X_test_s))`.

The red line in Figure 3 shows the result. The best test accuracy is at **k = 3: 113 of 114 correct, 99.1%**. From k = 5 onwards the accuracy settles at about 97.4%.

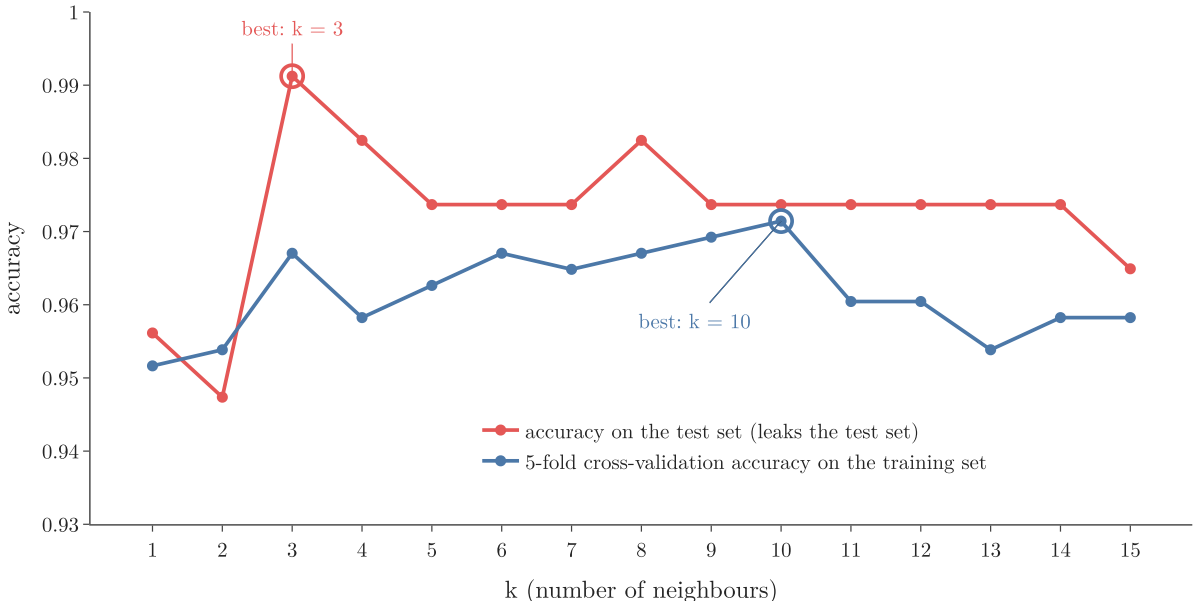


Figure 3: Accuracy against k on the breast cancer data. Red: scored on the test set. Blue: 5-fold cross-validation on the training set

Extra: Choosing k by its test-set score has a flaw: the test set is no longer unseen. We tried 15 models on the same 114 test observations and kept the luckiest, so the 99.1% is an optimistic score, not an honest estimate for new patients. Picking k on the test set is the same kind of leak as fitting the scaler on the test set.

The right way is **cross-validation** (G-510) on the training set only (the **pipelines** (G-1499), section 8): for each k we average the scores over 5 folds of the 455 training observations, keep the k with the best average, then score the test set **once**.

The blue line in Figure 3 is this cross-validation accuracy. Cross-validation picks $\mathbf{k = 10}$ (97.1% average), not 3. The final, honest test accuracy with k = 10 is **97.4%**.

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier())
grid = {"kneighborsclassifier__n_neighbors":
        list(range(1, 16))}
search = GridSearchCV(model, grid, cv=5)
search.fit(X_train, y_train)     # unscaled: the pipeline scales
search.best_params_              # k = 10
search.score(X_test, y_test)     # 0.974, the one test score
```

The scaler goes inside the pipeline so that, in every fold, it is fitted only on that fold's 4 training parts.

5. Decision surfaces

> **Key point:** A **decision surface** (G-560) colours every point of the input space by the class the model would predict there. The surface is built by predicting on a dense grid of points.

5.1 What it shows

> **Key point:** The coloured areas are the decision regions; the line where the colour changes is the decision boundary.

The **decision regions** (the coloured areas where one class is predicted; see how the model predicts) and the **decision boundary** between them (learning a decision boundary) are familiar. Drawn over the whole input space, they form a **decision surface** (G-560). A decision surface works for any classifier, as long as there are only 1, 2 or 3 features, so we can plot them.

To get two features, we use only the first two breast cancer measurements: `mean radius` and `mean texture`. Figure 4 shows KNN with k = 5 trained on them.

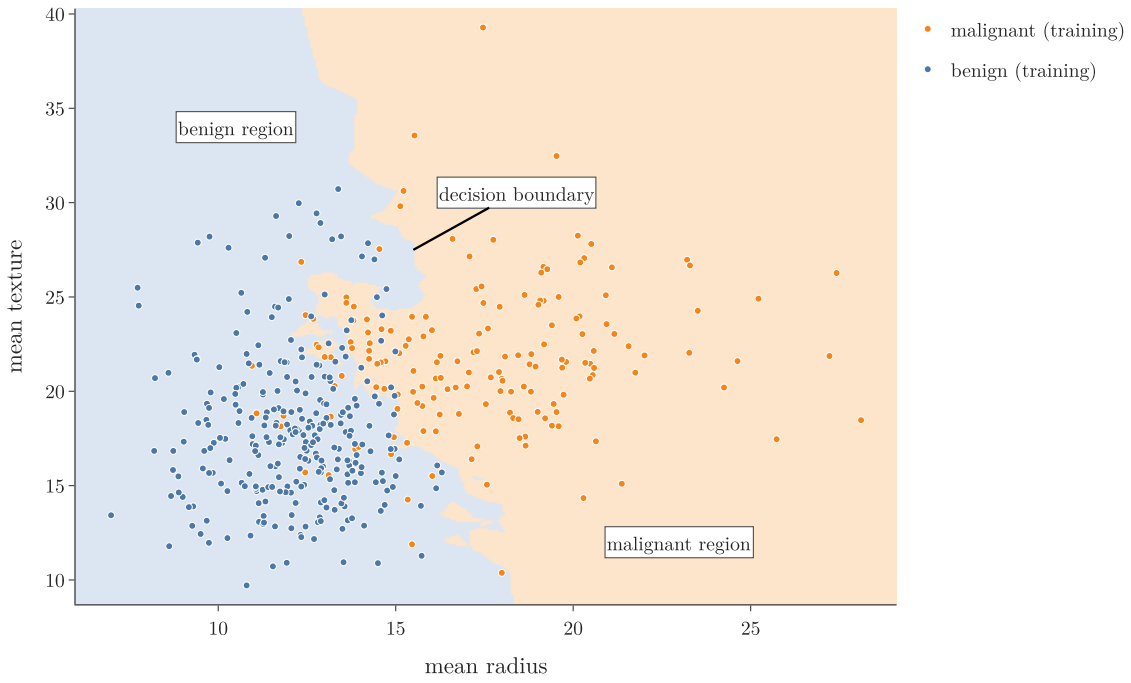


Figure 4: KNN (k = 5) on two breast cancer features: the decision regions and the decision boundary

The surface answers "what would the model predict here?" for every point at once. A new tumour with radius 12 and texture 15 falls in the blue region, so KNN predicts benign; one

with radius 20 falls in the orange region: malignant. KNN's boundary is not a straight line: it bends wherever the **neighbours** (G-1306) change.

5.2 How it is built

> **Key point:** Make a fine grid covering the data's range, predict the class of every grid point, and colour each one.

1. **Find the range** of each feature, with a small margin: here mean radius from about 6 to 29 and mean texture from about 9 to 40.
2. **Make a grid** of points covering that rectangle: 300 values on each axis give 90,000 points:

$$300 \times 300 = 90{,}000$$

   NumPy's `meshgrid` builds it.
3. **Predict** the class of every grid point with the trained model.
4. **Colour** each grid point by its predicted class (blue or orange) and draw the training points on top.

With 90,000 points packed together, the coloured dots merge into solid areas, and the boundary appears where blue meets orange.

Python: A **decision surface** (G-560) with NumPy and Plotly.

```
import numpy as np
import plotly.graph_objects as go

xs = np.linspace(X2[:, 0].min() - 1,
                 X2[:, 0].max() + 1, 300)
ys = np.linspace(X2[:, 1].min() - 1,
                 X2[:, 1].max() + 1, 300)
XX, YY = np.meshgrid(xs, ys)           # two 300 x 300 grids
grid = np.c_[XX.ravel(), YY.ravel()]  # 90,000 rows, 2 columns
Z = model.predict(grid).reshape(XX.shape)
go.Figure(go.Heatmap(x=xs, y=ys, z=Z))
```

`X2` holds the two features and `model` is a **pipeline** (G-1499) of `StandardScaler` and KNN trained on them. `meshgrid(xs, ys)` returns every combination of an x value and a y value: `XX` holds the x coordinates and `YY` the y coordinates. `ravel()` flattens each grid into one long column, `np.c_` puts the two columns side by side, and `reshape` folds the predictions back into a 300 by 300 grid for the heatmap.

6. How k controls overfitting and underfitting

> **Key point:** Small k follows every point, outliers included (overfitting); k = n always predicts the majority class (**underfitting** (G-2035)). A middle k is best.

Overfitting (G-1429) and underfitting, and their link to high variance and high bias, show up clearly in KNN. Figure 5 trains KNN on the same two features with four values of k.

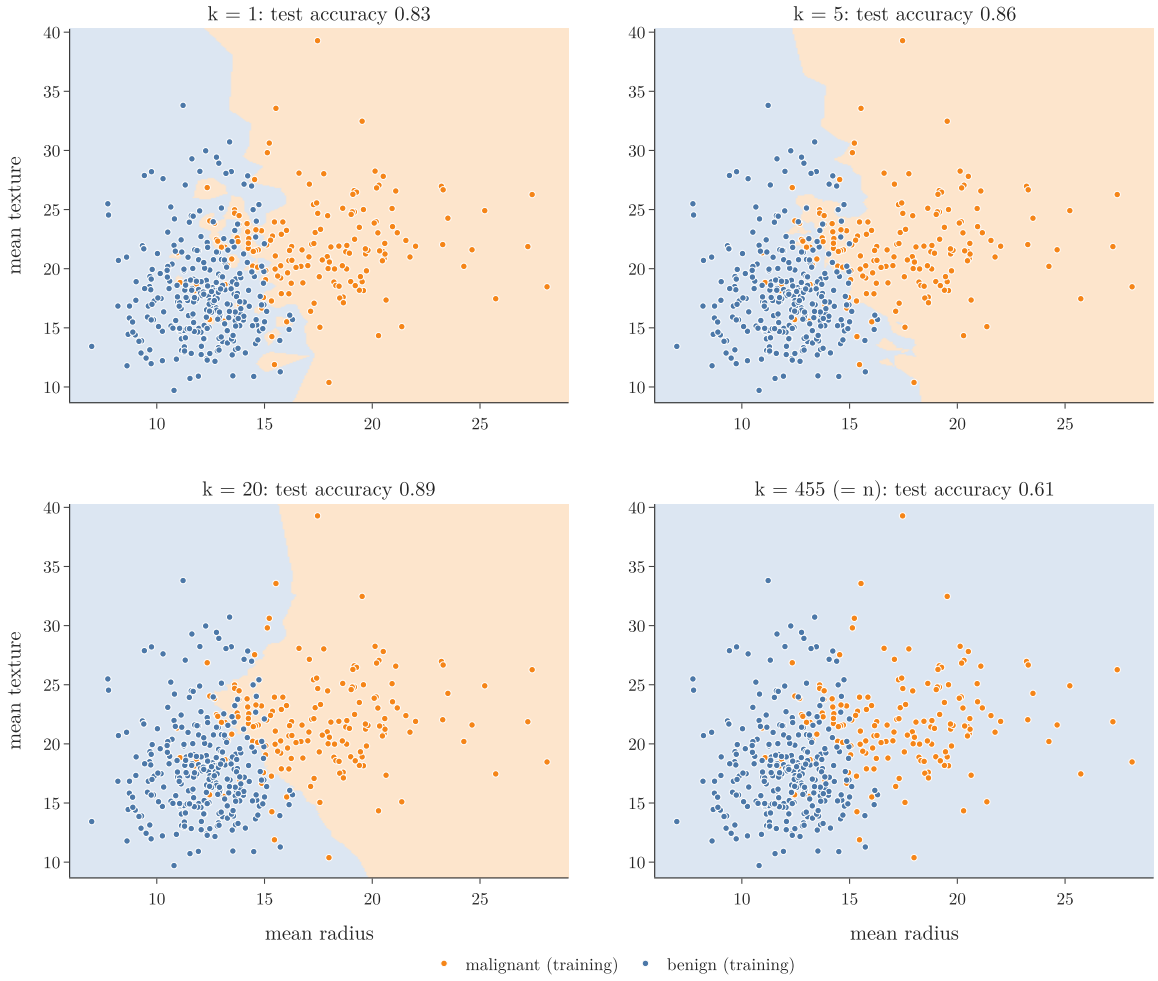


Figure 5: Decision surfaces for k = 1, 5, 20 and k = n = 455, with test accuracy on the same two features

Figure 6 sweeps k through 14 values from 1 to n. The accuracy panel on the right of each frame puts k on a log scale: the marked values 1, 3, 10, 30 and 100 each sit about three times further along than the one before, so small k values are spread out and the large ones are squeezed together. Watch the islands melt as k grows, and the test accuracy on the right rise, level off and then fall as the surface turns blue.

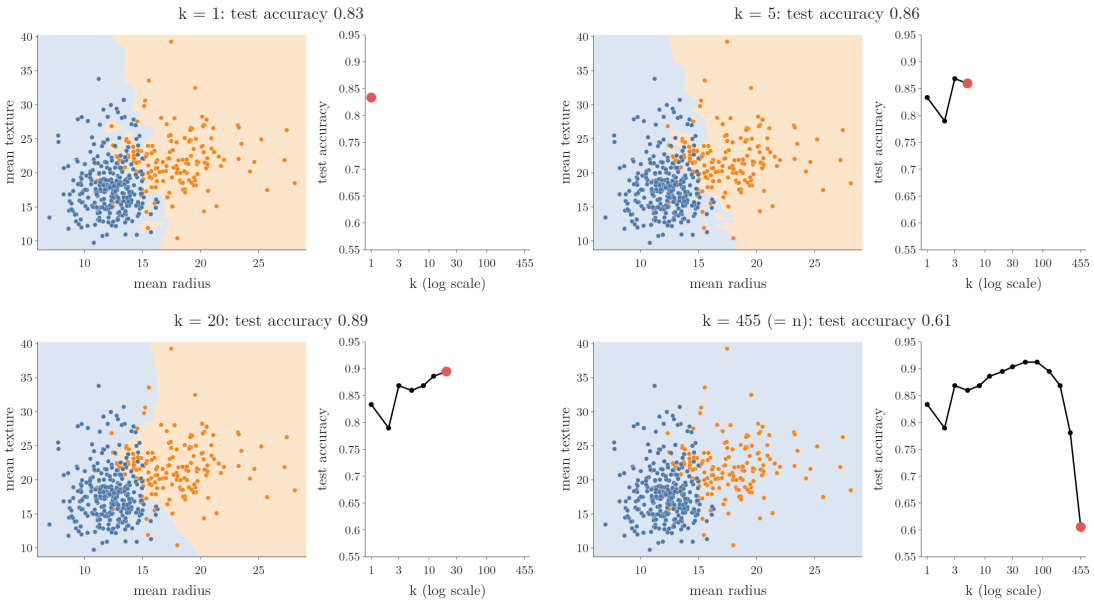


Figure 6: k swept from 1 to n = 455 on the same two features: the decision surface (left) and the test accuracy so far (right)

6.1 k = 1: overfitting

> **Key point:** With one **neighbour** (G-1306), every training point, even an outlier, claims a small region of its own.

With k = 1, the **query point** (G-1605) takes the class of its single nearest neighbour. Take a lone orange (malignant) point sitting among blue ones, an outlier. A query point next to it is copied as malignant, even though everything else around says benign.

So the surface for k = 1 (Figure 5, top left) is full of small islands: orange patches inside the blue region and blue patches inside the orange one. The model has memorised small accidents of this particular sample. Another sample would place the islands elsewhere, so predictions change a lot from sample to sample: **high variance**, **overfitting** (G-1429; ISL §2.2.3).

6.2 k = n: underfitting

> **Key point:** If k equals the number of training observations, every **query point** (G-1605) consults all of them, and the majority class always wins.

The largest possible k is n, the number of training observations; here n = 455. Then the "neighbourhood" of any query point is the whole training set, so the vote is always the same: 288 benign against 167 malignant.

The whole plane turns blue (Figure 5, bottom right): every tumour is predicted benign, wherever it lies. The model ignores the features entirely, which is **underfitting** (G-2035):

high bias (ISL §2.2.3). Its test accuracy, 0.61, is just the share of benign tumours in the test set.

6.3 The middle

> **Key point:** As k grows, the boundary smooths out and the islands vanish; past a point it becomes too simple.

At k = 5 the islands shrink, and at k = 20 the boundary is a smooth curve (Figure 5, bottom left). The k = 20 boundary is still not a straight line, but it follows the real shape of the data rather than individual points. On these two features, k = 20 also has the best test accuracy of the four (0.89).

So:

- **very small k** (1 or 2): **overfitting** (G-1429);
- **very large k** (up to n): **underfitting** (G-2035);
- **in between**: the best k, found by experiment as in section 4.

6.4 Trying it: the k slider app

> **Key point:** `app.py` redraws the **decision surface** (G-560) for any k from 1 to 455.

The folder of this Note contains `app.py`, a small Dash app. Run `python app.py` and open `http://127.0.0.1:8050`. Move the slider and the surface of Figure 5 is redrawn for that k, with its test accuracy.

Things to try:

- **k = 1, 2, 3:** count the islands around single points.
- **k = 5, 10, 20:** watch the islands disappear and the boundary smooth out.
- **k = 100, 200, 300:** the orange region shrinks as the larger benign class wins more votes.
- **k = 455:** everything is benign.

7. When KNN fails

> **Key point:** Six weak spots: big datasets, many features, outliers, unscaled features, imbalanced classes, and questions about which feature matters.

Knowing when not to use an algorithm matters as much as knowing how it works. KNN has six common failure cases. Unscaled features were handled in section 3.2: a feature in big numbers decides the distance alone, so we scale first. The other five follow.

7.1 Large datasets: slow predictions

> **Key point:** KNN is a lazy learner: training only stores the data, and every prediction computes a distance to every training observation.

KNN is **lazy learning** (G-1057; **instance-based learning**, G-955, see **instance-based learning** (G-955)): training only stores the points, and all the work waits for a query. For each query, KNN computes n distances, sorts them and votes.

So training is fast and prediction is slow. With 500,000 observations and 100 features, every single prediction needs 500,000 distances and a sort. On a website, a user who clicks "predict" and waits 3 seconds may simply leave; internet companies care a lot about **latency** (G-1048), the delay between a request and its answer.

Figure 7 measures the two times on random data with 30 features. "Training" on 500,000 observations takes about 28 milliseconds, while predicting 1,000 new observations takes about 720 milliseconds, and the prediction time grows with n. The exact times depend on the computer.

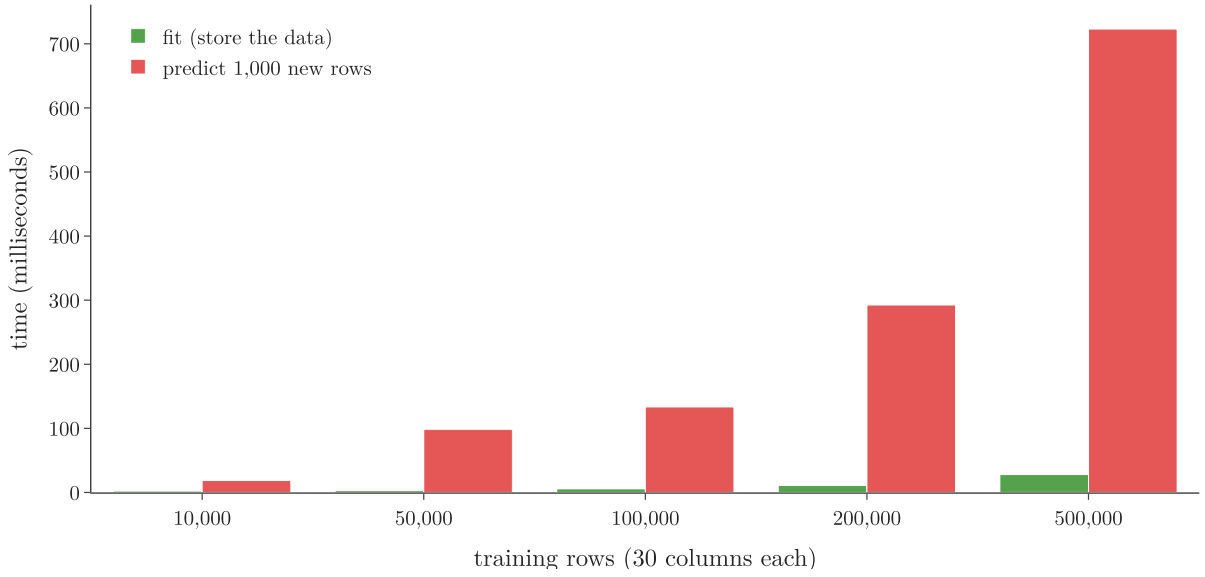


Figure 7: Fit time and prediction time of KNN (brute-force search) as the training set grows

> **Extra:** The measurement uses `algorithm="brute"`, the plain method described above. By default scikit-learn picks `algorithm="auto"`, which can store the points in a tree (a **KD-tree** or **ball tree**) so that most far-away points are skipped. That speeds things up a lot when there are few features (fewer than about 20), but helps little with many features, where the next failure case applies (scikit-learn user guide §1.6.4).

7.2 High-dimensional data: distances stop meaning much

> **Key point:** With many features, the curse of dimensionality makes all points nearly equally far apart, and KNN depends entirely on distances.

With many features, say 500, the **curse of dimensionality** (G-520; the **curse of dimensionality** (G-520)) sets in. In high dimensions the farthest point is barely farther than the nearest one (Figure 4 of that section: 67 times farther in 2 features, only 1.1 times in 1,000).

When every point is about equally far, the "nearest" **neighbours** (G-1306) are not really near, so their votes say little about the query point. KNN relies on nothing but distances, so once distances are distorted, its predictions are too (ISL §3.5). For very high-dimensional data, we either avoid KNN or first reduce the number of features.

7.3 Outliers

> **Key point:** With a small k, a few outliers carve out wrong regions around themselves.

Suppose two pink points sit deep inside the blue region: noise or outliers. With k = 3, a query point next to them has both as **neighbours** (G-1306) and is called pink. A pink island appears in the decision surface where the region should be blue.

The outlier island is the k = 1 picture of Figure 5 again: KNN is sensitive to outliers, and a k that is too small leads to **overfitting** (G-1429). A larger, well-chosen k outvotes them.

7.4 Imbalanced data

> **Key point:** If one class is rare, its points are usually outvoted, so predictions lean towards the common class.

In **imbalanced data** (G-921; see when accuracy misleads), one class is much rarer than the other, for example 98% "yes" and 2% "no". Around almost any query point, most **neighbours** (G-1306) belong to the common class, so the vote is biased towards it.

A quick test on synthetic data with 98% of observations in class 0 and 2% in class 1 (5,000 observations, 10 features) shows the effect (Figure 8). KNN with k = 5 scores 98% accuracy on the test set, yet finds only 1 of the 30 rare-class observations: a **recall** (G-1641) of 0.03. The accuracy looks excellent because almost everything is predicted as the common class.

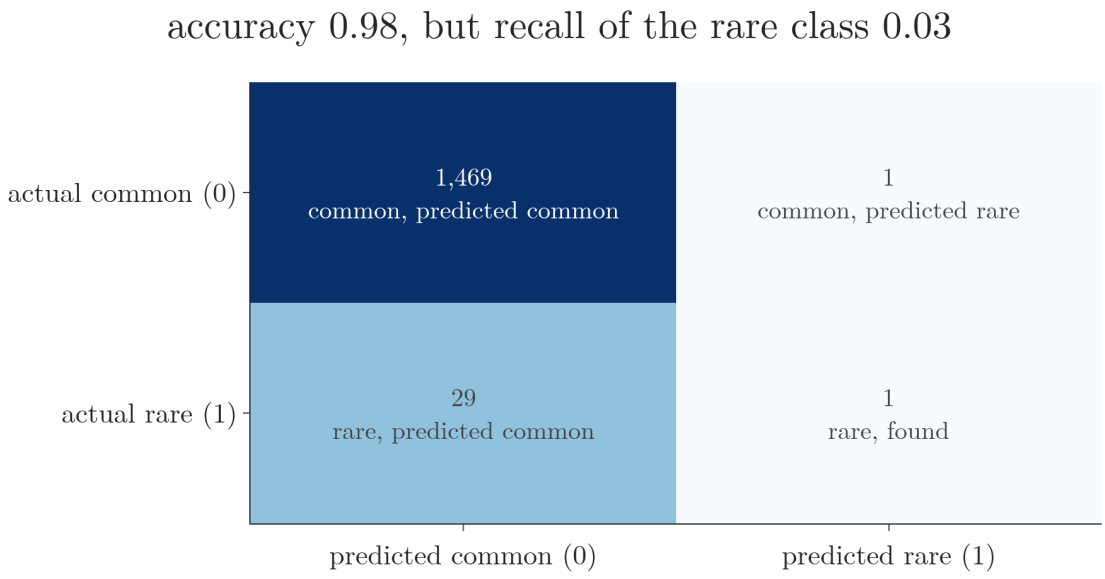


Figure 8: KNN (k = 5) on synthetic data with 2% rare class: test confusion counts. 1,469 common observations are right, but 29 of the 30 rare ones are predicted as common: accuracy 0.98, recall 0.03.

7.5 Inference: which feature mattered?

> **Key point:** KNN predicts well but does not say how much each feature contributes; it acts like a black box.

Sometimes we want **inference** (G-941) rather than prediction: we want to learn how the features affect the target. For example, does CGPA or IQ matter more for placement?

KNN cannot answer such a question. KNN labels a query point by its **neighbours** (G-1306), but never builds a function that shows how each feature moves the target. Linear or logistic regression, by contrast, gives one coefficient per feature, and can be preferred for that reason even when KNN predicts slightly better (ISL §3.5). For this purpose KNN acts as a **black box model** (G-311): it gives an answer, but not the reasons behind it.

8. Summary

Situation	KNN works well?	Why
Small or medium dataset	yes	few distances per prediction
Large dataset (hundreds of thousands of observations)	no	every prediction computes n distances: high latency
Many features (hundreds)	no	curse of dimensionality: distances lose meaning
Outliers, small k	no	islands around outliers: overfitting
Features on different scales	only after scaling	big-number features dominate the distance
Imbalanced classes	no	the common class wins most votes
Need to know which feature matters	no	no coefficients: a black box

- KNN predicts by majority vote among the k training points nearest to the query point, usually by Euclidean distance, because a point is probably like its **neighbours** (G-1306).
- Scale the features first: on the breast cancer data, 91.2% unscaled against 97.4% scaled (k = 5), because otherwise the features with big numbers decide the distance alone.
- Choose k by **cross-validation** (G-510) on the training set (here k = 10, test accuracy 97.4%), not by test-set accuracy, because picking the best of many test scores leaks the test set and gives an optimistic number. $\sqrt{n}$, made odd, is only a rough start, because k should grow with n yet stay a small share of it, but on real data of fixed size it often misses; odd avoids ties between two classes.
- A **decision surface** (G-560) colours a dense grid of predicted points (`meshgrid`, predict, colour), so you can see what the model would predict everywhere at once, for 1, 2 or 3 features.
- Small k overfits (islands, high variance), because every point, outliers included, claims a region of its own; k = n underfits (always the majority class), because every query consults the whole training set. So the best k lies in between.
- So KNN is a simple, often accurate classifier for small, scaled, low-dimensional data; the table above says when to reach for another model.

9. Sources

Built from

- CampusX, "What is K Nearest Neighbors? | KNN Explained in Hindi | Simple Overview in 1 Video | CampusX", YouTube, https://www.youtube.com/watch?v=abnL_GUGub4
- StatQuest with Josh Starmer, "StatQuest: K-nearest neighbors, Clearly Explained", YouTube, https://www.youtube.com/watch?v=HVXime0nQeI. The idea of placing a

new point among labelled points and letting its nearest neighbours vote (Figure 1); the growing circle is our own.

Other references

- **ISL:** James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd edition. Springer, 2021. Section 2.2.3, pp. 39–42 (KNN and the choice of K); Section 3.5, pp. 108–109 (curse of dimensionality, interpretability).
- **Anava and Levy 2016:** Anava, O. and Levy, K. Y. "k-*Nearest Neighbors: From Global to Local".* NeurIPS* 2016, arXiv:1701.07266. §1 (k-NN is consistent when k grows with n while k/n shrinks to 0; $k = \sqrt{n}$ is a widely mentioned rule of thumb that often gives poor results on finite samples, so k is usually chosen by cross-validation).
- **scikit-learn docs:** `sklearn.neighbors.KNeighborsClassifier` (metric, p); user guide Section 1.6.4, "Nearest Neighbor Algorithms", scikit-learn 1.9.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
K-nearest neighbours (KNN) (G-998)	Predicting from the answers of the k closest stored points.
Decision surface (G-560)	A plot colouring every point of the input space by the class the model would predict there.
Query point (G-1605)	The new point whose class we want to predict.
Majority vote (G-1146)	Predicting the class that most voters choose: the k neighbours in KNN, or the base models of an ensemble.
Minkowski distance (G-1227)	A family of distances between two points, $(\sum_i \lvert x_i - y_i \rvert^p)^{1/p}$, set by the number p: $p = 2$ gives the Euclidean distance and $p = 1$ the Manhattan distance; KNN uses it to find the nearest neighbours.
k (n_neighbors) (G-992)	The number of neighbours that vote; KNN's main hyperparameter.
Latency (G-1048)	The delay between a request and its answer; high for KNN on large data.
Inference (vs prediction) (G-941)	Learning how the inputs affect the output, rather than only predicting it; not the deployment sense (running a trained model) nor inferential statistics.

Term	Meaning
Black box model (black box) (G-311)	A model whose inner workings are hidden: it gives predictions without showing how each input contributed.
KNeighborsClassifier (G-1016)	scikit-learn's KNN classifier; `n_neighbors=5` by default.
meshgrid (G-1211)	NumPy function that builds every combination of x and y values: the grid for a decision surface.
Square-root rule (G-1862)	A rough starting value for k, the number of neighbours in KNN: about $\sqrt{n}$, where n is the number of training rows, made odd to avoid ties.
Nearest neighbours (neighbours) (G-1306)	The observations at the smallest distance from a given point; in KNN, the k training points closest to the query point, whose values or labels decide the prediction.
Instance-based learning (G-955)	Learning by storing the training data and comparing new points with it.
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Euclidean distance (G-715)	The straight-line distance between two points.
KNN imputer (G-1017)	Filling a missing value from the most similar rows (multivariate imputation, `KNNImputer`).
Standardization (G-1874)	Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable.
Accuracy (G-162)	The fraction of predictions that are correct.
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Pipeline (G-1499)	One object that chains preprocessing steps and a model, each step's output feeding the next; every input, including new ones after deployment, then gets exactly the same preprocessing.

Term	Meaning
Underfitting (G-2035)	Being too simple to capture the pattern; fails on all data.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Lazy learning (G-1057)	Another name for instance-based learning: no work until a question arrives.
Curse of dimensionality (G-520)	The problems that appear when data has too many dimensions: lower performance and more computation.
Imbalanced data (G-921)	Data in which one class is much rarer than another.
Recall (sensitivity) (G-1641)	Of all items that really are positive, the fraction the model found.